

Comparative Analysis of PyTorch vs fastai

Nichelle Graf

Team: Root Matrix

ITAI 2376 - Deep Learning in AI

Professor Rao Viswanatha

Introduction

PyTorch and fastai are closely related deep learning tools, but they operate at different levels of abstraction. PyTorch is a general-purpose deep learning framework developed for flexible, Python-native model development, while fastai is a higher-level library built on top of PyTorch. Rather than competing as completely separate frameworks, fastai extends PyTorch by simplifying common workflows and incorporating established deep learning practices into more accessible APIs.

1. Background and purpose

PyTorch grew out of Facebook AI Research and was released as open source in 2017. It became known for its flexible, Pythonic development model. It now supports everything from experimentation to production deployment, with distributed training, cloud support, accelerators, computer vision, NLP, reinforcement learning, and generative AI.

fastai was also introduced in 2017 by Jeremy Howard and Sylvain Gugger as a higher-level deep learning library built on PyTorch. It was explicitly designed as a layered API for deep learning, with the goal: give practitioners high-level tools that can produce strong results quickly, while still providing researchers and advanced users with lower-level components that can be modified or combined. (Howard & Gugger, 2020)

2. Usability

This is where fastai has the clearest advantage. Fastai, with roughly a few lines of code, can be used to create things like an image classifier, image-segmentation model, sentiment classifier, recommendation system, or tabular model. It deliberately abstracts away repetitive setup and incorporates modern training practices into its APIs.

PyTorch usually requires more explicit code and generally requires developers to define models, data loaders, optimization logic, and training loops more directly. The tradeoff is greater transparency and control. PyTorch gives the most direct control because fastai ultimately sits on top of PyTorch. That creates a nice continuum:

fastai high-level API → fastai lower-level components → PyTorch

Instead of two competing ecosystems, they can actually be used together.

3. Performance and scalability

Fastai relies on PyTorch for its underlying computational capabilities, so the two do not represent fundamentally different execution engines. fastai's advantage is primarily developer productivity: it allows practitioners to establish strong models and training workflows quickly. PyTorch, in contrast, provides more direct access to data loaders, sampling strategies, model components, and distributed-computing features. PathAI specifically cited PyTorch's configurable data loaders, flexible computational graphs, modular architecture, and ability to move rapidly from experimentation to scaled production as reasons it suited its pathology work.

PyTorch serves as a broad foundation for research and production systems across many areas of AI.

4. Support and ecosystem

PyTorch has a larger ecosystem by far. The PyTorch Foundation says its ecosystem includes more than 100 (120 in 2025) projects and supports areas ranging from computer vision and reinforcement learning to production tooling and cloud deployment (PyTorch Foundation, 2025).

Fastai has a smaller but highly focused ecosystem centered on accessible deep learning, practical training workflows, tutorials, a free course, and integrations with tools such as Hugging Face Hub, Captum, TensorBoard, Weights &

Biases, and others. Because fastai is built on PyTorch, users can begin with its high-level abstractions and move closer to native PyTorch when a project requires more customization.

5. Real-world applications

Fastai is used in research, industry, and education and provides specialized tools for medical imaging. Its official chest X-ray tutorial demonstrates a workflow for classifying pneumothorax using DICOM images, with high-level APIs handling image loading, training, prediction, and evaluation (fastai, n.d.). In contrast, PathAI uses PyTorch to build custom multiple-instance learning models for extremely large pathology slides. Its workflow divides whole-slide images into patches, generates representations, pools them into slide-level predictions, and preserves spatial interpretability (PyTorch, 2022). Both tools therefore support medical-imaging applications, but at different levels of abstraction: fastai emphasizes rapid application of established workflows, while direct PyTorch development provides greater control for specialized architectures and complex data-processing requirements.

A counterexample for PyTorch is PathAI's AI-powered pathology system. PathAI uses PyTorch to build custom multiple-instance learning (MIL) models for pathology slides. Their workflow involves selecting image patches from extremely large whole-slide images, generating patch representations, pooling them into slide-level predictions, and preserving spatial interpretability so clinicians can better understand why a model made a prediction.

Both PyTorch and fastai can support medical-imaging applications, but the level of abstraction differs. fastai is well suited to quickly applying established deep-learning workflows, while direct PyTorch development provides greater control when the problem requires specialized model architectures, data-processing strategies, or interpretability mechanisms.

Conclusion

PyTorch and fastai are complementary rather than direct competitors. PyTorch provides the flexible foundation, broad ecosystem, and fine-grained control required for custom research and production systems, while fastai builds on PyTorch to reduce complexity and accelerate model development through high-level APIs and built-in best practices. Developers who need maximum control may prefer working directly in PyTorch, while those who want rapid experimentation and strong baseline models with less boilerplate may benefit from fastai. Because fastai exposes lower-level components and interoperates with PyTorch, users can also begin at a high level and move toward more direct control as project requirements become more complex.

References

Howard, J., & Gugger, S. (2020). *fastai: A layered API for deep learning*. arXiv.

fastai. (n.d.). *Chest X-ray model*. fastai Documentation.

PyTorch. (2022). *Case study: PathAI uses PyTorch to improve patient outcomes with AI-powered pathology*. PyTorch.

PyTorch Foundation. (2025). *PyTorch Foundation expands to an umbrella foundation to accelerate AI innovation*. PyTorch.

AI Use Note:

I used generative AI to help organize my research, refine wording, and condense the report. I reviewed the final content and verified factual information using the cited sources.